



SYSTEM BIBLE · FULL LEADERSHIP PACK

Business + candidate + recruiter — one deck

Merged from Business, Candidate (Aarya), and Recruiter (Nitya) decks: product, stack, ingest, match math, agents, and honest status.

July 2026 · ← → · F fullscreen · Cmd/Ctrl+P → PDF · ~28 slides

ONE LINE

Not a job board with a chatbot

Candidates get Arya. Recruiters get Nitya. Both read/write the same Supabase graph.
Intros handshake via DB rows — never agent-to-agent RPC.

CANDIDATE

Aarya

Profile · chat/voice · Google Jobs ingest · scored matches · Request Intro

DB

intro_requests

RECRUITER

Nitya

JD · publish · score candidates · request intro to candidates (in-app) · Gmail cold path for HM intros

VALUE

Why it exists

Candidate

Less apply-spam. Scored roles from live ingest. One-tap intro from *their* Gmail.

Recruiter

JD in minutes (form or URL). Shortlist from the same matcher. Warm intros, not spray.

MVP FOCUS: MARKET IN

INR / LPA

OPT-IN DISCOVERY DEFAULT OFF

CODE SUPPORTS MULTI-MARKET LATER

What ships where

web/

Marketing · hireschema.com

Next.js 15 · Vercel

app/

Candidate + Recruiter SPA

/r/{slug} · Zustand · Realtime · SSE

api/

FastAPI · LangGraph

MatchingEngine · JobIngester · Aarya · Nitya

supabase/

Postgres · pgvector

Auth · Storage · RLS · NOTIFY

One deploy story

hireschema.com (web + app) → API (Railway → ECS target) → Supabase · CI: GitHub Actions

Confirmed technology

LAYER	CHOICE	NOTES
Frontend	Next.js 15 · TS · Tailwind · shadcn	web + app
Auth	Supabase · LinkedIn OAuth	Phone OTP not in active stack
API / agents	FastAPI · LangGraph · asyncpg	Single-threaded master loop
AI	OpenRouter · claude-sonnet-4.6 · gemini-2.5-flash	embedding-3-small · fallback required
Voice	Deepgram Nova-3 / Aura	Same chat surface
Jobs	Apify Google Jobs scraper	Actor johnvc/Google-Jobs-Scraper
JD import	Firecrawl + HTML/JSON-LD	Ashby/Greenhouse embeds
Email	Resend (txn) · Gmail OAuth (cold)	No SendGrid / MSG91 / Affinda on slides

How surfaces connect

APPS

Candidate SPA

Recruiter SPA

Marketing web/

API

FastAPI /api/v1

MatchingEngine

JobIngester

background_jobs

AGENTS

Aarya · LangGraph

Nitya · LISTEN

agent_actions → Realtime

DATA

Supabase Postgres + pgvector

Auth · Storage · Realtime

VENDORS

OpenRouter

Deepgram

Apify

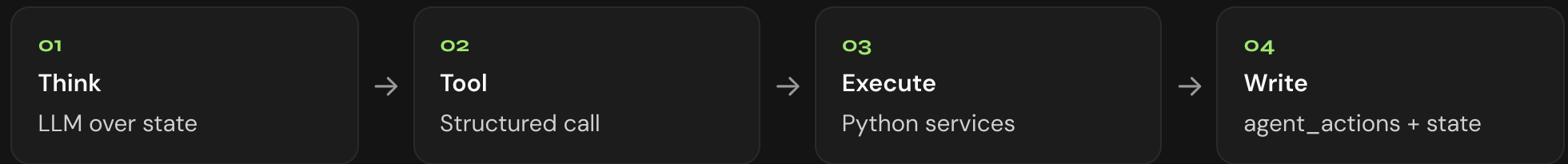
Firecrawl

Resend

Gmail OAuth

AGENTS

Master loop (both agents)



Intros: Request Intro → `intro_requests` → NOTIFY → Nitya. No Aarya↔Nitya HTTP.



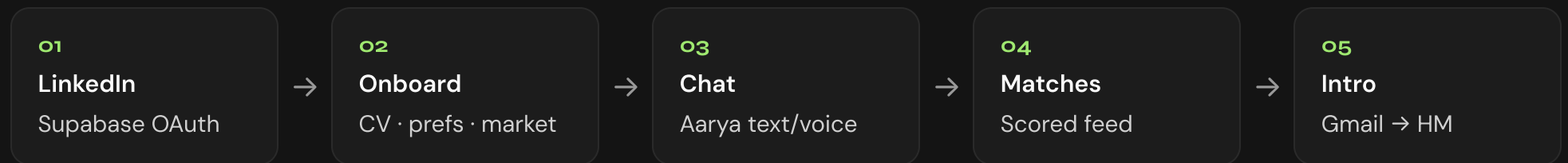
PART 2

Candidate - Aarya in detail

Journey, SPA, profile signals, Google Jobs ingest, match math, Request Intro.

CANDIDATE JOURNEY

A → Z



SPA + API paths

UI

/onboarding

/chat

/jobs · matches

/profile

/r/{slug}

CLIENT

api.ts → FastAPI only

Zustand

Realtime · agent_actions

API

routes/matches.py

agents/aarya/

JobIngester · MatchingEngine

profile_read

job_search

get_match_score

request_intro

update_profile

save_job

direct_apply

prepare_application_kit

analyze_resume

analyze_pasted_jd

build_career_path

Where candidate signals come from

LinkedIn OAuth

Identity + profile fields → users / candidates

Resume

Supabase Storage → in-app parser → skills / experience on candidate row

Aarya chat

Prefs, location, seniority, must-haves written back to state

Embeddings

candidate_embeddings: profile · skills · resume (1536-d, OpenRouter embedding-3-small)

services/embeddings.py · cosine via pgvector <=>

Where jobs come from

Apify · Google Jobs only

Actor: johnvc/Google-Jobs-Scraper (ApifyJobsScraper). LinkedIn/career-site kwargs may exist in config; ingest forces Google Jobs.

TRIGGERS

CRON /JOBS/INGEST

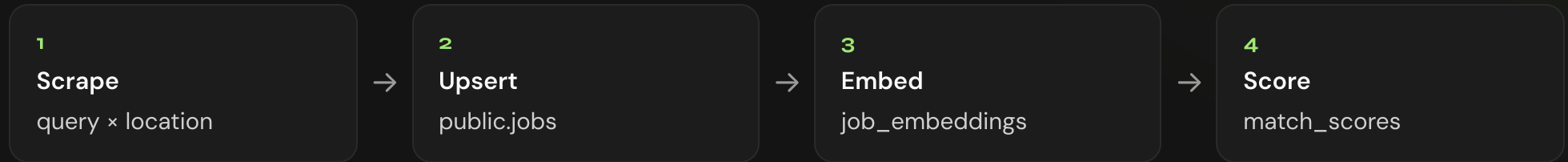
AARYA_AUTO_INGEST

POST /MATCHES/FIND-NEW

CAREER_PATH_INGEST

services/apify/jobs_scraper.py · job_ingester.py · routes/jobs.py · background_jobs.py

Scrape → jobs → embed → score



Dedup keys

apify_job_id → canonical_fingerprint → apply_url

Query dedupe

job_ingest_runs — skip same query/location within 24h unless force_refresh

Bucket freshness

Skip if bucket fresh 12h and inventory ≥ 8 (unless force)

Find new

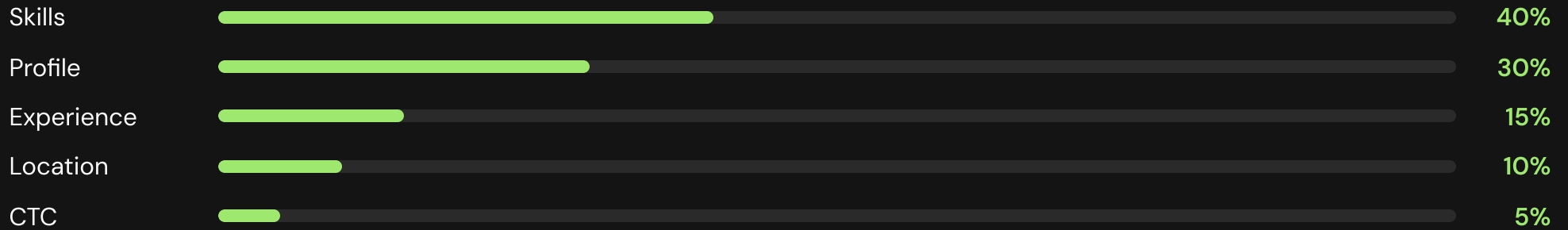
force_refresh=True; returns only jobs not in candidate_job_impressions

Embeddings are **not** inside JobIngestor — background workers after ingest.

How overall_score is calculated

Core math: MatchingEngine._assemble_score · services/matching.py

```
overall =  $\sum (w_i \times \text{dim}_i) / \sum w_{\text{included}}$   
# then  $\times$  gates (role · domain · title · seniority)  
  
w_skills = 0.40    w_profile = 0.30    w_experience = 0.15  
w_location = 0.10    w_ctc = 0.05  
# missing exp/CTC dims → weights renormed
```



Where each dim comes from

Skills (0.40)

Lexical: $0.85 \times \text{coverage} + 0.15 \times \text{jaccard}$. Then semantic lift from skills embeddings ($\alpha=0.40$) — embeddings only raise the lexical base.

Profile (0.30)

Title affinity + $\text{cosine}(\text{profile_embedding}, \text{jd_embedding})$ with calibrated lift ($\alpha=0.50$).

Experience (0.15)

Only if job seniority set — in-band 1.0, under/over linear decay.

Location (0.10)

Remote+open $\rightarrow 1.0 \cdot$ same city 1.0 $\cdot$ country/global 0.9 $\cdot$ state 0.7 $\cdot$ else 0.2

CTC (0.05)

Overlap vs job band (INR LPA); missing data $\rightarrow 0.5$

After the weighted mean

Gates (multipliers)

Role-fit · domain_fit · generic-title penalty ($\times 0.35$) · seniority gap ($1.0 \rightarrow 0.4$)

Quality floors

Persist ≥ 0.35 · feed ≥ 0.38 . Failed persist deletes old match_scores row.

RRF is for ordering – not the weight formula

RRF: $\text{score} += \text{weight} / (k + \text{rank} + 1)$ $k = 60$

Feed: fuse (overall, skills) $\rightarrow$ MMR $\lambda=0.72$ · company cap 2

Aarya job_search: $\text{RRF} + 0.08 \cdot \text{overall} + 0.04 \cdot \text{skills} + 0.02 \cdot \text{exp}$

matching.py · match_quality.py · ranking.py · job_recall_pipeline.py

Agents never call each other

CANDIDATE AGENT

Aarya

Request Intro writes a row — does not call Nitya.

SHARED BRIDGE

Postgres · intro_requests

- 1 **INSERT** → NOTIFY + background_jobs
- 2 pending → enriching → drafting → ready
- 3 Candidate reviews · Gmail if needed
- 4 **approve-send** → Gmail → sent...
- 5 Realtime → both UIs

HM / INTRO AGENT

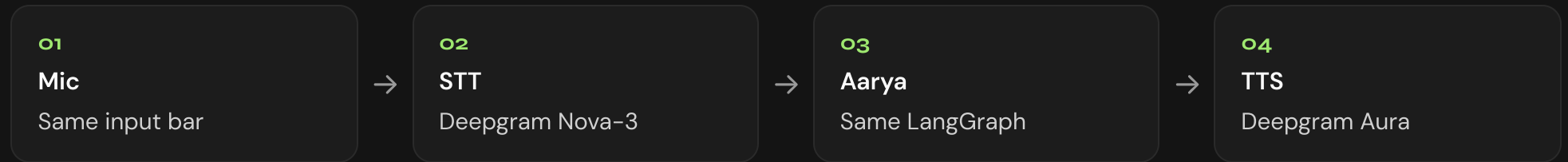
Nitya

LISTEN → enrich → draft → wait for approve-send.

✗ No Aarya ↔ Nitya HTTP / RPC — cold email = candidate Gmail; txn = Resend

VOICE · R8

Voice is the chat



No separate voice product. TTS timeout ~10s. Post-MVP: scheduled 15–20 min screening call (later slide).



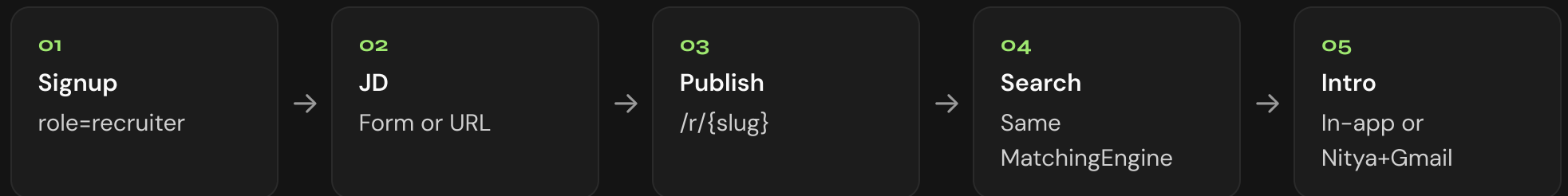
PART 3

Recruiter - Nitya in detail

JD import, publish, same MatchingEngine search, pipeline, intros, screening planned.

RECRUITER JOURNEY

A → Z



SPA + JD create / publish

UI

Roles / JD editor

Pipeline / search

Nitya chat

Inbox

API

routes/recruiter.py

POST /roles/{id}/search

JD URL extract

CORE

recruiter_search.py

MatchingEngine

agents/nitya/

MANUAL

Structured form

Title, location, remote, must/nice skills, comp, JD — zod / Pydantic.

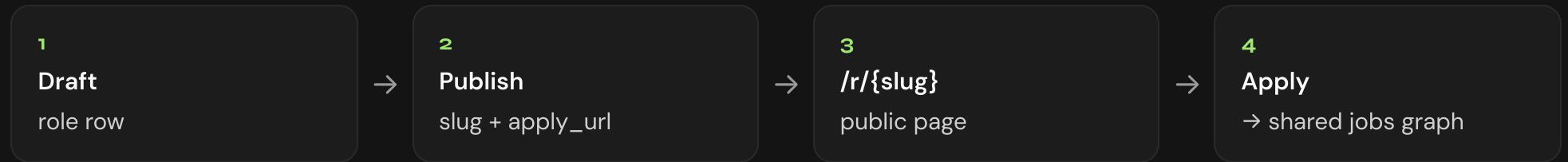
IMPORT

Career-page URL

Firecrawl → Ashby/Greenhouse → JSON-LD / HTML.

PUBLISH

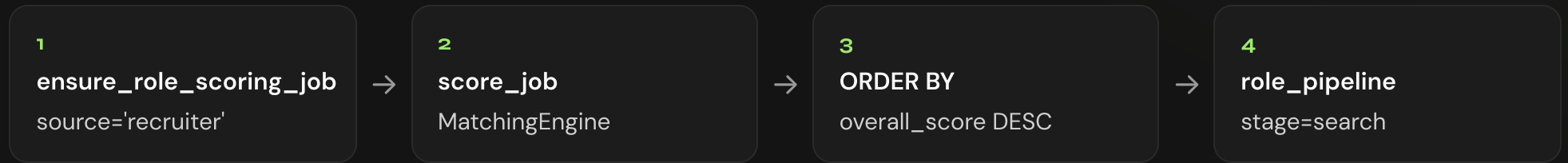
Draft → public apply link



Published roles enter the shared job graph under market visibility rules.

There is no separate graph ranker

“Find candidates” reuses the **same** MatchingEngine against a mirrored jobs row.



services/recruiter_search.py · POST /roles/{role_id}/search

How the mirror job is built

`ensure_role_scoring_job`

Find/create jobs by `role_id`, `source='recruiter'`. Sync title, JD, skills (must/nice), comp, location.

`score_job(job_id, limit=500)`

Identical `_assemble_score` vs eligible candidates.

Visibility

`share_with_recruiters = TRUE` and `visibility <> 'private'`.

Pipeline write

Upsert `role_pipeline` with `match_score + criterion_scores` JSON. Log `recruiter_searches`.

Per-criterion LLM scoring is reserved (P17+) — **not** live today.

overall_score on a role

```
overall = weighted dims → × gates
```

```
skills 0.40 · profile 0.30 · experience 0.15 · location 0.10 · ctc 0.05
```

```
skills_lex = 0.85×coverage + 0.15×jaccard (+ embedding lift)
```

Candidate vectors

candidate_embeddings: profile · skills · resume (1536-d HNSW)

Role / job vectors

job_embeddings: jd · title · skills — after sync / embed workers

Cards order by overall_score / pipeline match_score — not a second formula. Missing embeddings → lexical dims still run.

Intro path after shortlist

- **Recruiter → candidate intro:** in-app recruiter_to_candidate — candidate accepts/declines from inbox (Nitya agent does *not* run Gmail pipeline here)

- **Candidate → HM cold intro:** LISTEN on intro_requests · enrich · draft · approve—send via Gmail OAuth

- **Candidate → recruiter:** in-app inbox on the recruiter side

- **Enrichment (HM path):** Apify company employees → profile → email finder

Core tables

users

candidate | recruiter

candidates / recruiters

Profile sides

jobs + roles

Marketplace + recruiter JDs

match_scores

+ *_embeddings

intro_requests

Handshake SoR

role_pipeline

Recruiter kanban

agent_actions

Realtime counter

gmail_tokens

Cold send path

consent_log · background_jobs

DPDP · workers

TRUST

DPDP + hard rules

consent_log

Purpose + timestamp per collection event.

Export / delete

GET /me/export · DELETE /me soft-delete.

bias_audit

On every match_scores row.

PRIVACY@HIRESHEMA.COM

EMAIL COLD ONLY

RESEND TXN ONLY

NO AGENT RPC

Candid across product

Candidate

Match feed / Find new reliability · chat “find jobs” polish · ingest without embed workers → no scores

Recruiter

Search depends on role→job sync + embeddings · Gmail OAuth must be live · empty graph → empty shortlist

Shared deps

Apify · Gmail OAuth · Resend · embed/score workers. S01–S21 coded; E2E / S22 infra incomplete; payments deferred.

Next 2 weeks (stabilize MVP)

Match + Find new

Ingest→embed→score chain · impression hygiene · chat↔Matches parity

Intro + recruiter search

Gmail E2E · embed-after-role-sync · criterion breakdown · empty-search monitoring

Aarya 15–20 min screening call

After jobs + candidate + recruiter basics are stable — one Deepgram call, dual purpose.

CANDIDATE

Richer profile → better jobs

Exp · path · prefs → graph + embeddings → better MatchingEngine input.

RECRUITER

Screening on apply

Call insights + JD → screening card on pipeline (not a live interview).

Builds on today's Deepgram voice (S05). Does not replace MatchingEngine shortlist.

REMEMBER

Takeaways

- 01 Two agents, one graph — Aarya + Nitya share Supabase; intros via `intro_requests` only.
- 02 Jobs from Apify Google Jobs — scrape → upsert → embed → score.
- 03 One MatchingEngine — candidate feed and recruiter search use the same `overall_score` math.
- 04 Cold email = Gmail + **approve-send**; Resend stays transactional.
- 05 Screening call is post-MVP — after match/ingest stabilize.

Sources merged: 01-business · 02-candidate · 03-recruiter · system bible